BỘ GIÁO DỤC VÀ ĐÀO TẠO — KHOA CÔNG NGHỆ THÔNG TIN

MÔN HỌC: COMP1019 – LẬP TRÌNH TRÊN WINDOWS

BÁO CÁO BÀI TẬP LỚN & TÀI LIỆU THUYẾT TRÌNH

HỆ THỐNG QUẢN LÝ SẢN PHẨM (PRODUCT MANAGER)
WINDOWS FORMS NÂNG CAO & DATAGRIDVIEW

Chủ đề chuyên sâu: StatusStrip Real-time, Thống kê tồn kho & Đọc/Ghi File (SaveFileDialog)

Đơn vị thực hiện:	Nhóm 3 – Lớp COMP1019
Nền tảng công nghệ:	C# .NET 9.0 / Windows Forms / Visual Studio 2022
Trạng thái mã nguồn:	Đã tối ưu giao diện gọn gàng, biên dịch hoàn tất (0 Warning, 0 Error)
Thời điểm hoàn thiện:	Tháng 10 / 2026

Tài liệu phục vụ thuyết trình báo cáo, phản biện đồ án và hướng dẫn giải thích chi tiết mã nguồn

I. TỔNG QUAN NHIỆM VỤ NHÓM 3 (THEO YÊU CẦU ĐỀ BÀI)

Theo phân công đề tài Lab 06, **Nhóm 3** chịu trách nhiệm nghiên cứu chuyên sâu và xây dựng giải pháp hoàn thiện cho 3 chủ đề kỹ thuật cốt lõi:

Chủ đề nghiên cứu	Yêu cầu kỹ thuật	Triển khai thực tế trong mã nguồn
1. Cập nhật StatusStrip theo thời gian thực (Real-time)	Hiển thị trạng thái hành động vừa thực hiện ("*Vừa thêm SP...*", "*Đã xóa...*") và tổng số lượng sản phẩm. Cập nhật đồng hồ hệ thống từng giây.	Sử dụng `ToolStripStatusLabel` (với thuộc tính `Spring = true`) phối hợp `System.Windows.Forms.Timer` (chu kỳ 1000ms).
2. Chỉ số thống kê tồn kho	Tính toán tổng giá trị tồn kho theo công thức: $\text{Tổng giá trị} = \sum (\text{Số lượng} \times \text{Đơn giá})$ và hiển thị trực tiếp lên thanh StatusStrip.	Duyệt qua danh sách `_bindingSource` để tính tổng tiền tức thời, tự động cập nhật khi thêm/sửa/xóa hoặc khi lọc danh mục.
3. Xuất file bằng SaveFileDialog & StreamWriter	Cho phép người dùng chọn vị trí lưu và xuất toàn bộ danh sách sản phẩm ra tệp tin `.txt` căn lề báo cáo hoặc `.csv` mở trên Excel.	Xây dựng lớp `ProductFileService`, sử dụng mã hóa `UTF8Encoding(true)` (BOM) để chống lỗi font tiếng Việt trên Microsoft Excel.

Sản phẩm & Yêu cầu thuyết trình:

- **Demo trực tiếp:** Minh họa thanh StatusStrip cập nhật tự động sau mỗi thao tác (Thêm, Sửa, Xóa, Lọc) và chức năng Xuất file ra đĩa cứng.
- **Báo cáo lý thuyết:** Trình bày rõ ràng luồng xử lý I/O ghi file, kiến trúc Two-way Data Binding và công thức tính tổng tồn kho từ danh sách dữ liệu.

II. TẠI SAO DÙNG NÓ? SO SÁNH CHỨC NĂNG & TÍNH THỰC TIỄN

Để đạt điểm tối đa trong phần phản biện với Giảng viên, nhóm làm rõ sự khác biệt giữa các giải pháp truyền thống (sơ sài) và các kỹ thuật chuyên nghiệp được áp dụng trong đồ án:

1. So sánh StatusStrip với MessageBox và Label thường

Tiêu chí đánh giá	`MessageBox.Show()`	`Label` thường đặt trên Form	✅ `StatusStrip` (Nhóm 3)
Chặn luồng (Modal Blocking)	Có – buộc người dùng dừng thao tác để click nút OK.	Không chặn luồng.	**Không chặn luồng:** Thông điệp hiển thị mượt mà, người dùng tiếp tục làm việc bình thường.
Vị trí chuẩn UI Windows	Hộp thoại nhảy ra giữa màn hình.	Đặt rải rác tùy ý, dễ gây rối mắt.	**Nằm cố định ở đáy cửa sổ** theo chuẩn UX của các phần mềm chuyên nghiệp (VS Code, Excel, Word).
Hiển thị đa thông tin cùng lúc	Chỉ hiển thị được 1 đoạn văn bản duy nhất.	Phải tạo nhiều Label rời rạc, khó căn chỉnh.	**Chia thành nhiều ô độc lập**: Thông báo hành động, Mặt hàng, Tổng tồn kho, Đồng hồ thời gian thực.
Khả năng co giãn (Responsive)	Không co giãn.	Phải tự viết mã Anchor/Dock phức tạp.	Tự động co giãn theo chiều rộng cửa sổ nhờ thuộc tính `Spring = true`.
Tính thực tiễn	Chỉ phù hợp cho lỗi nghiêm trọng hoặc xác nhận.	Chỉ dùng cho nhãn tiêu đề tĩnh.	**Chuẩn mực công nghiệp** cho việc phản hồi trạng thái liên tục trong ứng dụng quản trị.

2. So sánh SaveFileDialog với Đường dẫn cố định (Hardcode Path)

Tiêu chí	Đường dẫn cố định (VD: `C:\output.txt`)	✅ Sử dụng `SaveFileDialog` (Nhóm 3)
Tính linh hoạt	Cố định 1 vị trí; file luôn bị ghi đè nếu không đổi tên.	**Tự do hoàn toàn:** Người dùng chủ động chọn thư mục lưu trữ và đặt tên file theo ý muốn.
Bảo mật & Phân quyền	Dễ văng lỗi `UnauthorizedAccessException` khi ghi vào ổ C:\ hoặc Desktop máy khác.	Windows đảm bảo an toàn phân quyền vì chính người dùng đã chọn vị trí lưu hợp lệ.
Đa dạng định dạng (Filter)	Chỉ xuất đúng 1 định dạng đã viết cứng trong mã.	Hỗ trợ chuyển đổi linh hoạt giữa `.txt` và `.csv` qua bộ lọc Filter.
Cảnh báo ghi đè	Không cảnh báo, dễ làm mất dữ liệu quan trọng của người dùng.	Tự động hiển thị hộp thoại xác nhận *"Tệp đã tồn tại, bạn có muốn ghi đè không?"*.

3. So sánh StreamWriter với File.WriteAllText()

Tiêu chí	`File.WriteAllText()`	✅ `StreamWriter` trong khối `using`
Kiểm soát mã hóa (Encoding)	Mặc định UTF-8 không BOM, khi mở file CSV bằng Microsoft Excel sẽ bị **lỗi font tiếng Việt**.	Chủ động thiết lập `new UTF8Encoding(true)`, ghi đúng 3 byte BOM (`EF BB BF`) giúp **Excel hiển thị tiếng Việt hoàn hảo**.
Cơ chế ghi dữ liệu	Phải gom toàn bộ văn bản vào 1 biến String khổng lồ trong RAM rồi mới ghi xuống đĩa.	Ghi trực tiếp từng dòng dữ liệu qua luồng Stream đệm (Buffer), tiết kiệm bộ nhớ RAM tối đa.
An toàn tài nguyên (Resource Leak)	Nếu có lỗi giữa chừng, khó kiểm soát việc giải phóng Handle tệp tin.	Cú pháp `using` tương đương `try-finally`, tự động gọi `Flush()` và `Dispose()` trong mọi tình huống.

4. So sánh BindingList<Product> với List<Product>

Trong Windows Forms, nếu liên kết DataGridView với `List<Product>` thông thường, mỗi khi thêm, sửa hoặc xóa dữ liệu, DataGridView **hoàn toàn không tự động cập nhật**. Lập trình viên buộc phải gán lại `DataSource = null` rồi gán lại danh sách, gây giật lag và mất vị trí cuộn trang.

Ngược lại, `BindingList<Product>` hỗ trợ cơ chế **Two-way Data Binding**. Khi danh sách thay đổi, nó lập tức kích hoạt sự kiện `ListChanged`, DataGridView tự động vẽ lại chính xác dòng bị tác động mà không cần can thiệp thủ công.

III. THIẾT KẾ GIAO DIỆN TINH GỌN (APP PRODUCT MANAGER ĐƠN GIẢN HÓA)

Để phần mềm vừa đáp ứng đầy đủ 100% tính năng theo đề bài, vừa mang phong cách hiện đại, thanh thoát và hiển thị hoàn hảo trên các màn hình máy chiếu độ phân giải vừa phải, nhóm đã thực hiện tái cấu trúc giao diện:

1. Bảng so sánh giao diện Ban đầu vs. Giao diện Tinh gọn mới

Hạng mục giao diện	Phiên bản cồng kềnh ban đầu	✅ Phiên bản Tinh gọn (Mới nhất)
Kích thước cửa sổ	1220 × 704 px (Quá lớn, dễ tràn màn hình laptop 14 inch)	**900 × 630 px** (Vừa vặn tuyệt đối, gọn gàng, cân đối)
Thanh Header tiêu đề	Panel xanh đậm chiếm 70px chiều cao gây lãng phí không gian	**Loại bỏ hoàn toàn**; thông tin đặt gọn vào thanh tiêu đề TitleBar của Form
Khung hiển thị hình ảnh	Panel to 320 × 218 px làm phình to khung nhập liệu	PictureBox **135 × 110 px đặt gọn gàng góc phải**, vẫn đảm bảo phóng to zoom ảnh
Kích thước các nút bấm	105 × 38 px (Cồng kềnh)	**80 – 100 × 34 px** (Thanh thoát, hiện đại, phối màu trực quan)
Độ dài thông báo StatusStrip	Dài dòng: "📦 *Mặt hàng: 5 | Tổng SL tồn: 85 chiếc*"	Ngắn gọn, súc tích: "📦 *5 mặt hàng | SL: 85*"
Số dòng mã Designer	944 dòng	**568 dòng** (Giảm 40% mã nguồn thừa, tốc độ nạp Form cực nhanh)

2. Sơ đồ luồng trạng thái an toàn (Form State Machine)

QUY TRÌNH CHUYỂN ĐỔI 3 TRẠNG THÁI FORM

1. CHẾ ĐỘ XEM (VIEW)
Khóa toàn bộ chi tiết
Vô hiệu nút 'Cập nhật' & 'Hủy'
Bật Thêm, Sửa, Xóa, Xuất File

➔

2. CHẾ ĐỘ THÊM (ADD)
Xóa trắng dữ liệu ô nhập
Mở khóa toàn bộ thông tin
Bật nút 'Cập nhật' & 'Hủy'

➔

3. CHẾ ĐỘ SỬA (EDIT)
Mở khóa thông tin chi tiết
Khóa Mã SP (Khóa chính)
Bật nút 'Cập nhật' & 'Hủy'

Sau khi bấm **"Cập nhật"** (Lưu thành công) hoặc **"Hủy bỏ"**, hệ thống tự động đưa Form trở lại **Chế độ Xem (View)** để bảo vệ dữ liệu.

IV. GIẢI THÍCH CHI TIẾT TOÀN BỘ CODE `FORM1.DESIGNER.CS` (DÒNG 1 ĐẾN HẾT)

File `Form1.Designer.cs` gồm **568 dòng mã** được chia thành 10 khối chức năng logic rõ ràng:

Phần 1 (Dòng 1 – 15): Khai báo Partial Class & Cơ chế giải phóng bộ nhớ (Dispose)

- `partial class Form1`: Từ khóa `partial` cho phép tách đôi mã nguồn của lớp `Form1` thành 2 tệp: `Form1.cs` (chứa code nghiệp vụ) và `Form1.Designer.cs` (chứa code giao diện). Trình biên dịch C# sẽ tự động gộp lại thành một lớp duy nhất khi build.
- `private System.ComponentModel.IContainer components = null;`: Khai báo thùng chứa (container) quản lý các thành phần phi giao diện (chẳng hạn như Timer) cần theo dõi vòng đời.
- `protected override void Dispose(bool disposing)`: Phương thức giải phóng các tài nguyên không quản lý (Unmanaged Resources như handle GDI+, bút vẽ, phông chữ). Khi Form bị đóng, nó gọi `components.Dispose()` để tránh rò rỉ bộ nhớ (Memory Leak).

Phần 2 (Dòng 16 – 81): Hàm InitializeComponent, Khởi tạo đối tượng & Tạm dừng vẽ layout

- `private void InitializeComponent()`: Điểm khởi tạo toàn bộ giao diện, được gọi ngay trong hàm khởi tạo `Form1()`.
- Dòng 21 – 69: Khởi tạo mới (`new`) toàn bộ 35 đối tượng điều khiển (GroupBox, TextBox, ComboBox, NumericUpDown, DateTimePicker, PictureBox, Button, DataGridView, StatusStrip, Timer).
- Dòng 71 – 80 (`SuspendLayout()` và `BeginInit()`): Ra lệnh cho hệ điều hành Windows **tạm dừng cơ chế vẽ lại màn hình** trong lúc thiết lập hàng trăm thuộc tính. Kỹ thuật này giúp Form khởi động cực nhanh và triệt tiêu hoàn toàn hiện tượng nhấp nháy giao diện (Flickering).

Phần 3 (Dòng 82 – 105): Thiết lập GroupBox thông tin sản phẩm (grpThongTin)

- `grpThongTin.Dock = DockStyle.Top`: Neo cố định nhóm thông tin ở phía trên cùng của cửa sổ, tự động dàn trải chiều ngang khi co dãn màn hình.
- `grpThongTin.Size = new Size(900, 210)`: Chiều cao được thu gọn xuống 210px (bản cũ là 245px), tạo thêm không gian rộng rãi cho bảng DataGridView bên dưới.
- `grpThongTin.Controls.Add(...)`: Thêm toàn bộ các điều khiển con (nhãn, ô nhập liệu, ảnh, cụm nút bấm) vào bên trong khung.

Phần 4 (Dòng 106 – 202): Cấu hình các ô nhập liệu (Hàng 1 & Hàng 2)

- **Hàng 1 (Dòng 107 – 146):**
  - `txtMaSP`: `ReadOnly = true` (mặc định khóa), `PlaceholderText = "VD: SP06"` hướng dẫn nhập.
  - `txtTenSP`: Ô nhập tên sản phẩm, kích thước 210px.
  - `cboLoaiSP`: `DropDownStyle = DropDownList` (chỉ cho phép chọn từ danh sách, không cho gõ linh tinh), gồm 7 danh mục: Điện thoại, Laptop, Máy tính bảng, Phụ kiện, Linh kiện, Thiết bị mạng, Khác.
- **Hàng 2 (Dòng 148 – 202):**

- `numDonGia` : `Maximum = 1,000,000,000` (1 tỷ VNĐ), `ThousandsSeparator = true` (hiển thị phân cách hàng nghìn như 29,990,000 VNĐ), căn phải số học.
- `numSoLuong` : `Maximum = 100,000`, căn phải số học.
- `dtpNgayNhap` : `CustomFormat = "dd/MM/yyyy"` định dạng ngày tháng theo chuẩn Việt Nam.
- `chkConKinhDoanh` : CheckBox màu xanh lá (`DarkGreen`), mặc định là Checked (đang kinh doanh).

Phần 5 (Dòng 203 – 218): Khung hình ảnh thu nhỏ (PictureBox) & Nút chọn ảnh

- `picHinhAnh` : Kích thước **135 × 110 px** đặt tại tọa độ (750, 20) góc phải trên. `SizeMode = PictureBoxSizeMode.Zoom` giúp ảnh luôn giữ đúng tỷ lệ khung hình, không bị méo mó.
- `btnChonAnh` : Nút "📷 *Chọn ảnh*" đặt ngay dưới PictureBox, mặc định bị Disable và chỉ sáng lên khi bấm Thêm SP hoặc Chỉnh sửa.

Phần 6 (Dòng 219 – 301): Cụm 7 nút bấm thao tác (pnlButtons)

Sử dụng `FlowLayoutPanel` giúp các nút tự động dồn hàng từ trái qua phải ngay ngắn:

- `btnThem` (90 × 34 px): Màu xanh lá `RGB(46, 125, 50)` – Mở form thêm mới.
- `btnChinhSua` (95 × 34 px): Màu cam `RGB(230, 126, 34)` – Mở khóa sửa dòng đang chọn.
- `btnCapNhat` (100 × 34 px): Màu xanh dương `RGB(21, 101, 192)` – Mặc định **Disabled**, chỉ sáng khi Thêm/Sửa.
- `btnHuy` (80 × 34 px): Màu xám ghi `RGB(120, 144, 156)` – Mặc định **Disabled**, hủy bỏ thao tác.
- `btnXoa` (80 × 34 px): Màu đỏ `RGB(198, 40, 40)` – Xóa sản phẩm kèm hộp thoại xác nhận.
- `btnLamMoi` (90 × 34 px): Màu xám đậm `RGB(69, 90, 100)` – Khôi phục toàn bộ danh sách gốc.
- `btnXuatFile` (100 × 34 px): Màu tím `RGB(106, 27, 154)` – Mở hộp thoại xuất file TXT/CSV.

Phần 7 (Dòng 302 – 359): Khung danh sách (grpDanhSach) & Thanh tìm kiếm/Lọc nhanh (pnlSearch)

- `grpDanhSach.Dock = DockStyle.Fill`: Tự động chiếm trọn vẹn toàn bộ phần diện tích còn lại ở giữa cửa sổ.
- `txtTimKiem`: Gắn sự kiện `TextChanged += txtTimKiem_TextChanged` – Lọc kết quả real-time ngay khi gõ từng ký tự.
- `cboLocLoai`: Gắn sự kiện `SelectedIndexChanged` – Lọc theo nhóm danh mục tức thì.
- `btnBoLoc`: Nút "Bỏ lọc" giúp xóa từ khóa tìm kiếm và đưa ComboBox về vị trí "-- *Tất cả loại* --".

Phần 8 (Dòng 360 – 450): Bảng hiển thị dữ liệu DataGridView (dgvSanPham)

- `AutoGenerateColumns = false`: Tắt tính năng tự sinh cột để kiểm soát 100% thứ tự, kiểu dáng và độ rộng từng cột.
- `ReadOnly = true` và `SelectionMode = FullRowSelect`: Khóa không cho người dùng sửa trực tiếp trên ô, click chọn là chọn trọn vẹn cả hàng.
- Header màu xanh navy sang trọng (`RGB(37, 64, 97)`), dòng xen kẽ sọc xanh nhạt (`RGB(245, 248, 252)`) chống mỏi mắt.
- 8 cột dữ liệu liên kết chuẩn qua `DataPropertyName`: Mã SP (70px), Tên SP (AutoFill chiếm trọn không gian trống), Loại SP (110px), Đơn giá (130px - Format N0), SL Tồn (75px), Thành tiền (140px - Format N0), Ngày nhập (100px - dd/MM/yyyy), và Cột Checkbox Kinh doanh (45px).

Phần 9 (Dòng 451 – 482): Thanh trạng thái StatusStrip & Timer đồng hồ

- `statusStrip1.Dock = DockStyle.Bottom`: Luôn nằm cố định ở chân Form.
- `lblTrangThai`: Thiết lập `Spring = true` để tự động giãn rộng sang bên phải, hiển thị thông điệp thao tác hiện tại.
- `lblTongSanPham`: Chữ đậm xanh navy báo cáo tổng số mặt hàng và tổng lượng tồn kho.
- `lblTongGiaTriTonKho`: Chữ đậm màu đỏ `DarkRed` làm nổi bật tổng giá trị quy đổi thành tiền của toàn bộ kho.
- `lblDongHo` & `timerDongHo`: Timer chu kỳ 1000ms kích hoạt sự kiện `timerDongHo_Tick` để hiển thị ngày giờ chuẩn xác theo thời gian thực.

Phần 10 (Dòng 483 – 568): Cấu hình Form tổng thể & Khai báo biến thành viên (Fields)

- `ClientSize = new Size(900, 630)` và `MinimumSize = new Size(800, 560)`.
- `StartPosition = FormStartPosition.CenterScreen`: Mở cửa sổ ngay giữa màn hình khi chạy.
- Dòng 498 – 511 (`ResumeLayout()` và `PerformLayout()`): Bật lại cơ chế vẽ màn hình và hoàn tất sắp đặt bố cục.
- Dòng 516 – 568: Khai báo toàn bộ 35 biến điều khiển `private` để file logic `Form1.cs` có thể truy cập và tương tác.

V. KỊCH BẢN THUYẾT TRÌNH BÀI BẢN & CÂU HỎI PHẢN BIỆN THƯỜNG GẶP

1. Kịch bản thuyết trình 7 bước (Demo trực quan trên máy)

Bước	Lời thoại người thuyết trình	Thao tác thực hiện trên máy tính	Điểm nhấn cần chỉ cho Giảng viên
1	*"Chào Thầy/Cô, nhóm em xin demo phần mềm ProductManager với giao diện tinh gọn và chuyên sâu Nhóm 3."*	Chạy ứng dụng từ Visual Studio.	Form mở lên 900×630 cực kỳ vừa vặn. Toàn bộ ô chi tiết bị KHÓA, nút Cập nhật bị Disable để chống sửa nhầm.
2	*"Khi click các dòng trên DataGridView, dữ liệu và ảnh minh họa được nạp lên để XEM CHI TIẾT."*	Click chọn lần lượt các dòng sản phẩm trên bảng.	Thông tin nhảy lên các ô nhập, PictureBox đổi ảnh tương ứng. Các ô hoàn toàn ở trạng thái chỉ đọc.
3	*"Quan sát StatusStrip dưới cùng: Hiển thị trạng thái 'Sẵn sàng', tổng 5 mặt hàng, tổng tồn kho và đồng hồ chạy từng giây."*	Chỉ chuột vào thanh StatusStrip dưới chân Form.	Nhãn trạng thái dạt trái (Spring=true), tổng tiền tính đúng bằng $\sum(\text{SL} \times \text{Đơn giá})$.
4	*"Bây giờ em bấm '➕ Thêm SP': Hệ thống xóa trắng ô nhập, MỞ KHÓA các trường và SÁNG nút 'Cập nhật'."*	Bấm ➕ **Thêm SP** → Nhập SP06, Tên, Đơn giá → Bấm 💾 **Cập nhật**.	Lưu thành công, sản phẩm vào DataGridView, các ô TỰ ĐỘNG KHÓA LẠI, nút Cập nhật tắt, tổng tồn kho StatusStrip nhảy tức thì!
5	*"Chức năng 'Chỉnh sửa': Chỉ khi ấn nút 'Chỉnh sửa' thì mới mở khóa thông tin để sửa lại."*	Chọn SP01 → Bấm 🖍️ **Chỉnh sửa** → Sửa đơn giá → Bấm **Cập nhật**.	Mã SP được giữ nguyên, thông tin lưu lại và Form tự động quay về chế độ Xem an toàn.
6	*"Demo Tìm kiếm và Lọc danh mục theo thời gian thực."*	Gõ chữ "iphone" hoặc chọn loại "Điện thoại" → Bấm "Bỏ lọc".	Bảng lọc tức thì; StatusStrip tính tổng tồn kho riêng cho nhóm đang lọc. Bấm Bỏ lọc khôi phục toàn bộ.
7	*"Tính năng trọng tâm Nhóm 3: Xuất báo cáo bằng SaveFileDialog & StreamWriter."*	Bấm 📤 **Xuất File** → Chọn file .txt và .csv → Mở file kiểm tra.	Notepad mở lên bảng biểu căn lề thẳng hàng; Excel mở file CSV hiển thị tiếng Việt có dấu chuẩn 100% không vỡ font.

2. Câu hỏi phản biện thường gặp của Giảng viên & Câu trả lời mẫu

Câu hỏi 1: Tại sao bình thường nhóm lại khóa hết các ô chi tiết và chỉ mở nút Cập nhật khi bấm Thêm hoặc Sửa?

Trả lời: Đây là nguyên tắc *Form State Management* (Quản lý trạng thái giao diện) trong các hệ thống chuyên nghiệp. Nếu luôn mở ô nhập liệu, người dùng khi click xem chi tiết rất dễ vô tình chạm bàn phím làm thay đổi sai lệch dữ liệu gốc. Việc khóa mặc định và chỉ mở khi người dùng xác nhận ý định (qua nút Thêm hoặc Sửa) giúp bảo vệ tính toàn vẹn dữ liệu cao nhất.

Câu hỏi 2: Tại sao khi ghi file bằng StreamWriter nhóm lại đặt trong khối `using (...)`?

Trả lời: Khối `using` trong C# là cú pháp viết gọn của cấu trúc `try-finally` kết hợp phương thức `Dispose()`. Nó đảm bảo kể cả khi quá trình ghi file gặp sự cố ngắt quãng hoặc lỗi đột ngột, luồng ghi file vẫn chắc chắn được đẩy hết dữ liệu đệm xuống đĩa (`Flush`) và đóng tệp (`Close`), không bao giờ bị tình trạng tệp bị Windows khóa cứng.

Câu hỏi 3: Làm thế nào để file CSV khi mở trực tiếp bằng Microsoft Excel không bị lỗi font tiếng Việt?

Trả lời: Microsoft Excel mặc định khi mở file CSV sẽ đọc theo bảng mã ANSI/Windows-1252 nếu không thấy ký hiệu nhận diện. Để khắc phục, nhóm em khởi tạo luồng ghi bằng `new StreamWriter(filePath, false, new UTF8Encoding(true))`. Tham số `true` yêu cầu tự động chèn 3 byte tiền tố Byte Order Mark (BOM - `0xEF, 0xBB, 0xBF`) vào đầu tệp, giúp Excel nhận diện ngay lập tức là mã UTF-8 và hiển thị tiếng Việt hoàn hảo.